

CSE145 - Introduction to Computing

Assignment 2: Turing Machines - Design, Analysis & Simulation

Name: Muhammad Wasay Farrukh

ERP No.: 35538

Personalization block: $R = 38$ $V = 2$ $B = 100110$ $n = 7$

Working: $R =$ last two digits of 35538 = 38. $V = 38 \bmod 4 = 2$. $B = 38$ in binary = 100110. Sum of digits = $3+5+5+3+8 = 24$, $24 \bmod 5 = 4$, so $n = 4 + 3 = 7$.

Variant ($V = 2$): Q1 $M = 5$, Q2 $a^{2n} b^n$, Q3 x^2 (append a 0), Q4 even-length palindromes over $\{a,b\}$, Q5 $\{a^i \# b^j : i > j\}$.

GitHub repository: <https://github.com/wasaay/35538-cse145-a2-tm>

All machines were built and run in AutomataVerse as single-tape machines. For every Task C run the repository has the JSON export and the screenshot (same file name, in the folder for that question), and README.md has the AutomataVerse share link for each run.

Q1 - Divisible-by-M Recognizer (V = 2, so M = 5)

Task A - Strategy

My machine reads the binary number from left to right and only keeps track of the remainder of the number read so far when it is divided by 5. There are only five possible remainders (0 to 4), so I used one state for each: q0 means remainder 0, q1 means remainder 1, and so on up to q4. When the head reads the next bit, the number so far becomes (old number x 2 + bit), so the new remainder is $(2r + \text{bit}) \bmod 5$ and the machine moves to that state. The head does not change the tape, it writes back the same bit and moves right each time. When it reaches the blank after the last bit it goes to the accept state q5 if it is in q0, and to the reject state q6 from any other state. The empty string never leaves q0, so it gets accepted as the number 0.

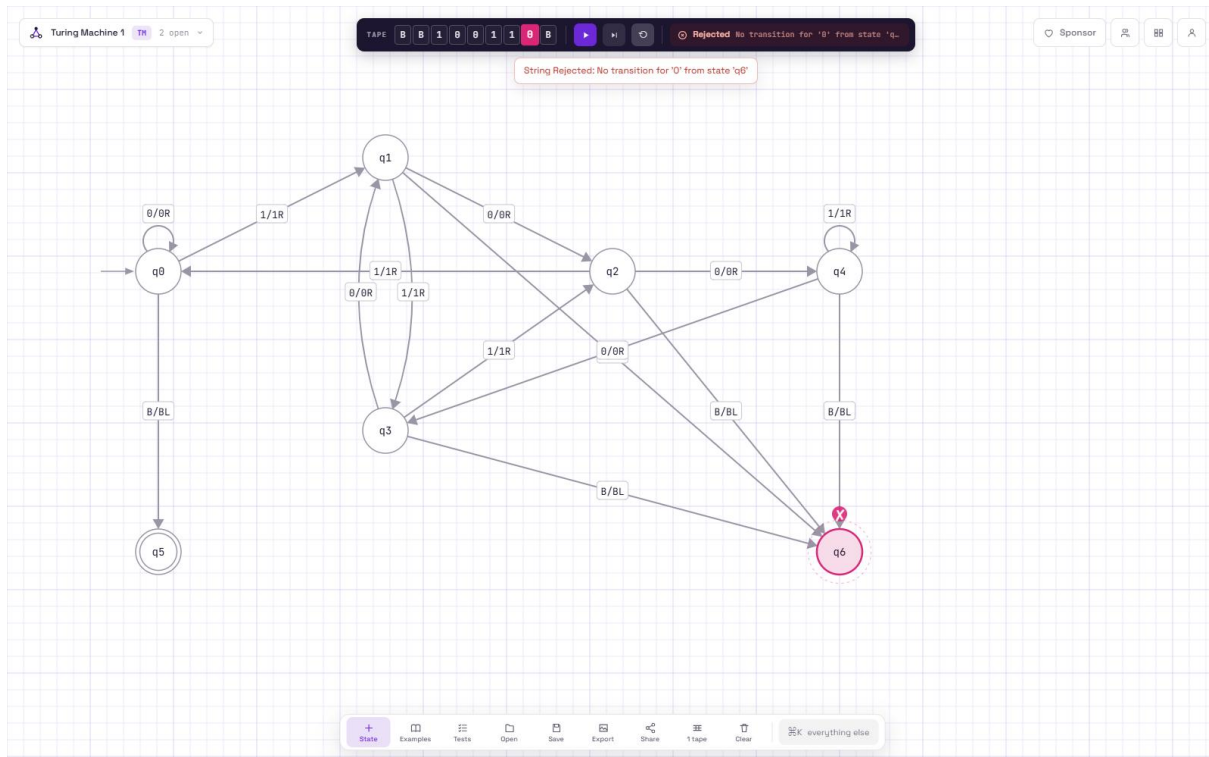
Task B - Transition table and state diagram

In AutomataVerse the blank symbol is B and the start state is always q0.

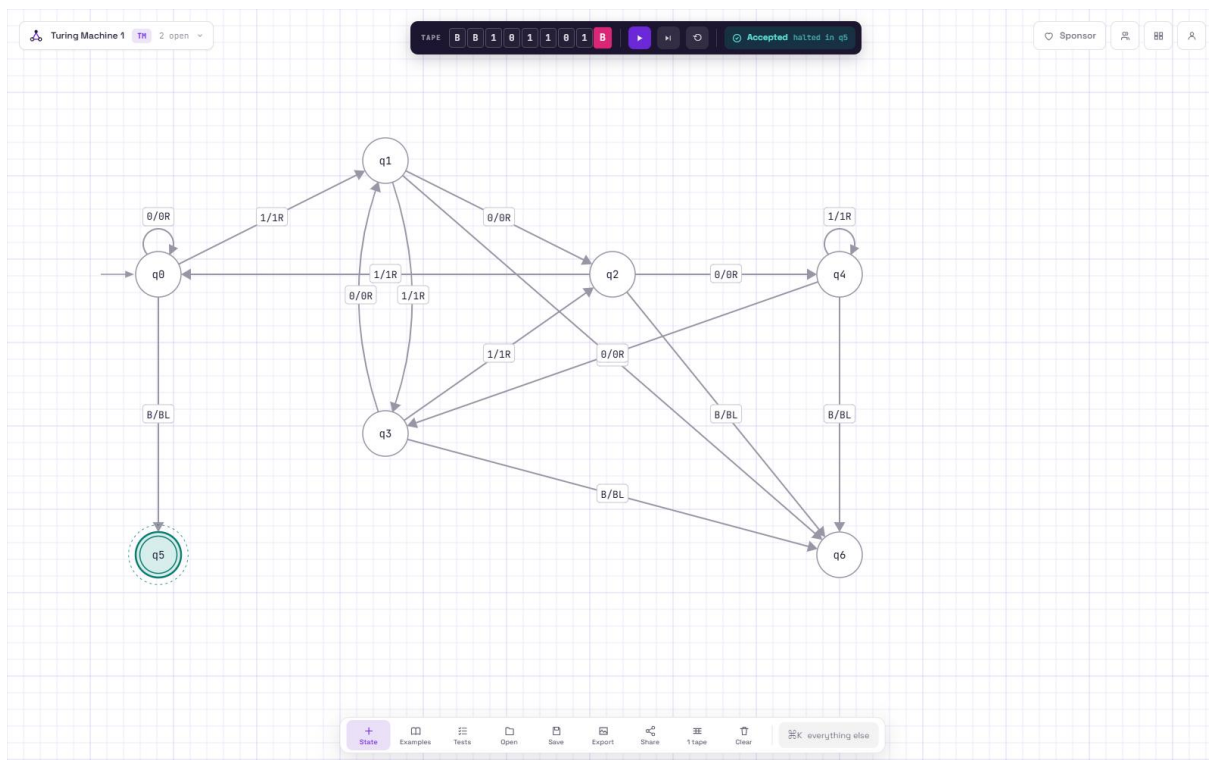
State	Meaning
q0 (start)	remainder 0 so far
q1	remainder 1
q2	remainder 2
q3	remainder 3
q4	remainder 4
q5	accept (halting)
q6	reject (halting)

Current state	Read	Write	Move	Next state
q0	0	0	R	q0
q0	1	1	R	q1
q0	B	B	L	q5
q1	0	0	R	q2
q1	1	1	R	q3
q1	B	B	L	q6
q2	0	0	R	q4
q2	1	1	R	q0
q2	B	B	L	q6
q3	0	0	R	q1
q3	1	1	R	q2
q3	B	B	L	q6
q4	0	0	R	q3
q4	1	1	R	q4
q4	B	B	L	q6

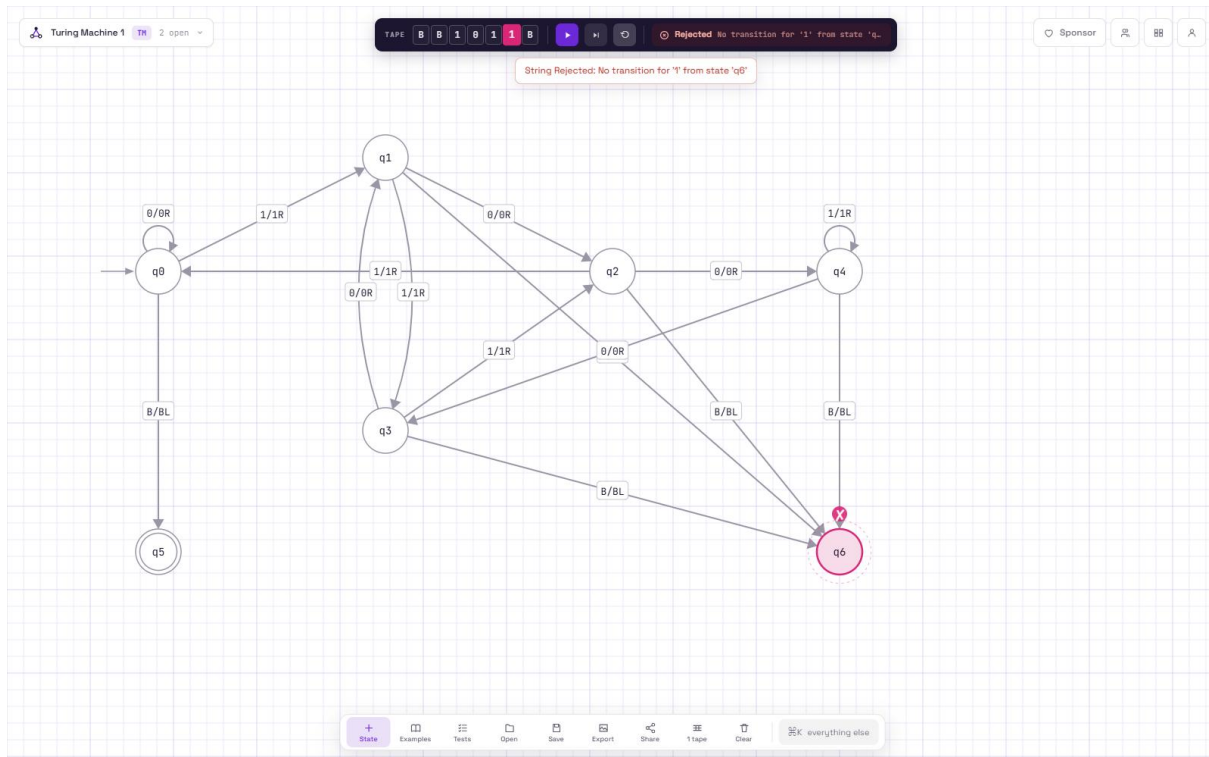
q5 and q6 have no outgoing transitions, so the machine halts as soon as it enters them. The diagram below is the same machine as drawn in AutomataVerse (q5 is the double circle).



Run 1: input 100110 (B), rejected, halted in q6



Run 2: input 101101, accepted, halted in q5



Run 3: input 1011, rejected, halted in q6

Note: when the machine halts in q6 AutomataVerse shows the message "No transition for ... from state q6". That is just how it reports a halt in a non-accepting state, q6 is my reject state.

Task D - Why the machine only works for $M = 5$

The value 5 is built into the states themselves. My machine has exactly five remainder states and every arrow was worked out using $(2r + \text{bit}) \bmod 5$, for example q2 on 1 goes to q0 only because $2 \times 2 + 1 = 5$. For another M I would need M remainder states instead of 5, and every transition would have to be recalculated using mod M , so basically the whole table changes and only the general idea of tracking the remainder stays the same.

Q2 - Scaled-Count Recognizer ($V = 2$, so $L = \{ a^{2n} b^n \}$)

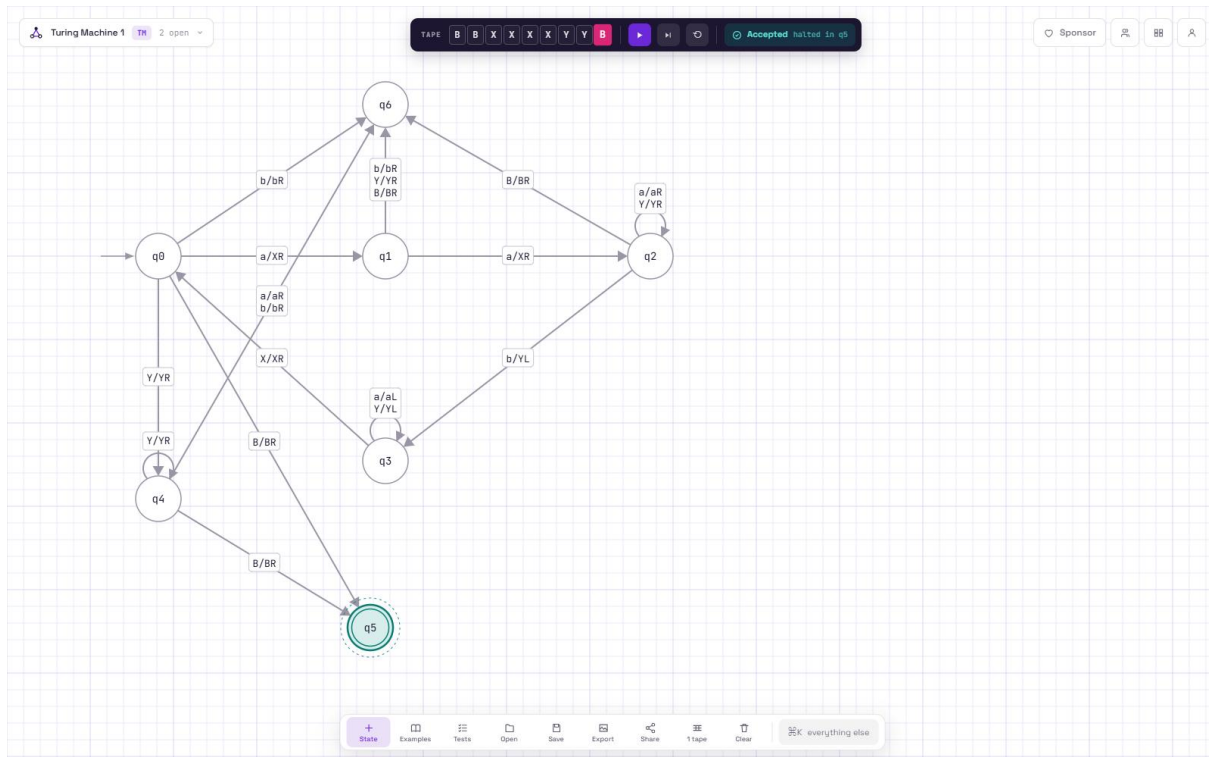
Task A - Strategy

The idea is to match two a's with one b in every round. Starting from the left, the machine crosses off the first unmarked a by writing X (q0 to q1), then crosses off the next a with an X too (q1 to q2). In q2 it moves right over the remaining a's and any Y's until it finds the first b that is not marked yet, and changes it to Y. Then q3 walks back left over a's and Y's until it hits the last X, steps right and is in q0 again for the next round. When q0 sees a Y instead of an a, all the a's have been used, so the machine goes to q4 and checks that there is nothing but Y's up to the blank. If it gets to the blank it accepts in q5. If it ever finds something it does not expect (a b when it needs the second a, a blank while looking for a b, or a leftover a or b in q4) it goes to the reject state q6. I took $n \geq 0$, so the empty string is accepted straight from q0.

Task B - Transition table and state diagram

State	Meaning
q0 (start)	cross the 1st a of a pair, or start the final check
q1	cross the 2nd a of the pair
q2	go right to the first unmarked b
q3	go back left to the last X
q4	final check, only Y's allowed
q5	accept
q6	reject

Current state	Read	Write	Move	Next state
q0	a	X	R	q1
q0	Y	Y	R	q4
q0	B	B	R	q5
q0	b	b	R	q6
q1	a	X	R	q2
q1	b	b	R	q6
q1	Y	Y	R	q6
q1	B	B	R	q6
q2	a	a	R	q2
q2	Y	Y	R	q2
q2	b	Y	L	q3
q2	B	B	R	q6
q3	a	a	L	q3
q3	Y	Y	L	q3
q3	X	X	R	q0
q4	Y	Y	R	q4
q4	B	B	R	q5
q4	a	a	R	q6
q4	b	b	R	q6



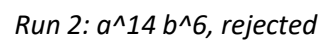
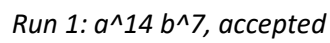
State diagram in AutomataVerse (screenshot from run 3)

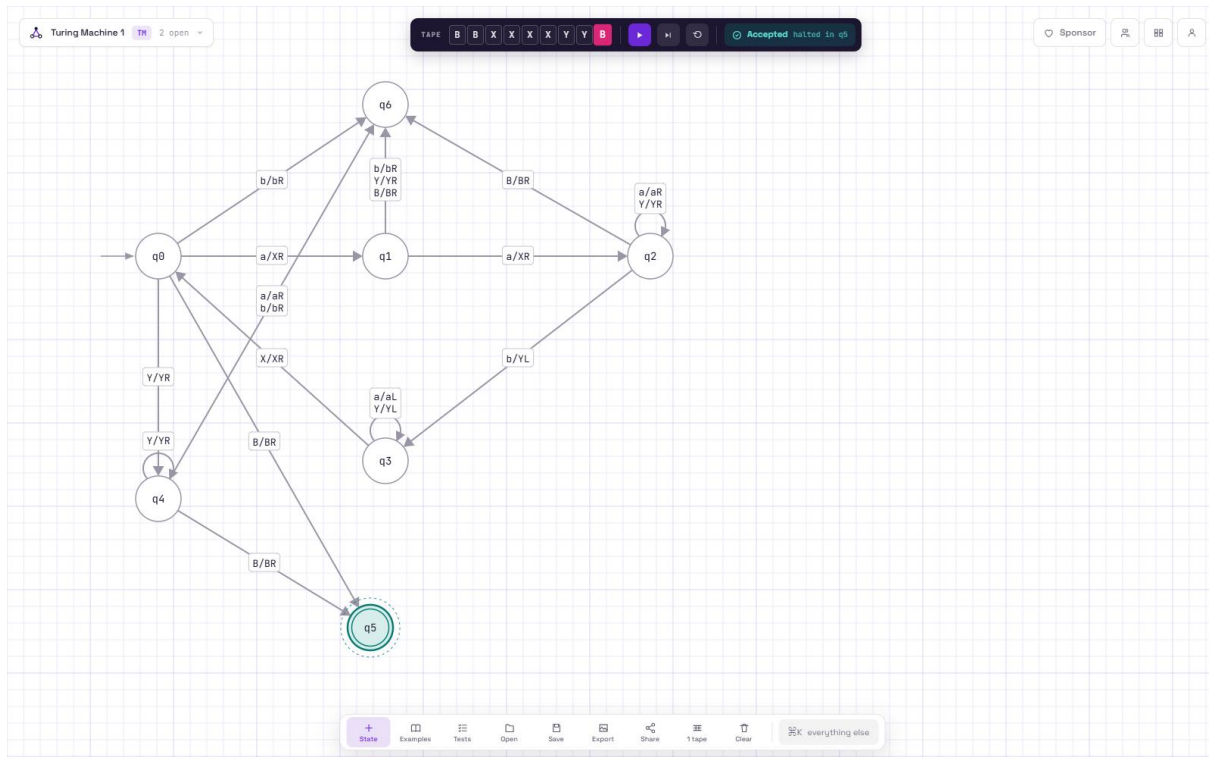
Task C - Simulator runs

$n = 7$, so my string is $a^{14} b^7$. Files are in the Q2 folder and links in README.md (lines "Q2 Run 1/2/3").

Run	Input	Result	Correct?	Files
1	aaaaaaaaaaaaabbbbbbb (14 a, 7 b)	Accepted, halted in q5, tape XXXXXXXXXXXXXXXXYYYYYYY	Yes, $14 = 2 \times 7$	q2_run1_n7.json / .png
2	aaaaaaaaaaaaabbbbbbb (one b removed, 14 a, 6 b)	Rejected, halted in q6	Yes, 14 is not 2×6	q2_run2_one_b_removed.json / .png
3	aaaabb ($n = 2$)	Accepted, halted in q5	Yes, $4 = 2 \times 2$	q2_run3_own.json / .png

In run 2 the first six rounds use up 12 a's and all 6 b's. In round 7 the machine crosses the last two a's but q2 then finds a blank instead of a b, so it goes to q6.





Run 3: aaaabb, accepted

Task D - Where an off-by-one error could happen

The easiest place to get it wrong is the $q_0 \rightarrow q_1 \rightarrow q_2$ part, which is what makes the ratio 2 to 1. If I only had one "cross an a" state the machine would accept $a^n b^n$ instead, and if q_1 did not reject when it sees a Y, an odd number of a's could get through. The other spot is the end: when q_0 sees a Y the machine has to make sure there are no a's or b's left, which is what q_4 does. To check it I traced a few small strings by hand: aab is accepted, aaab is rejected because q_1 reads Y after the first round, and aabb is rejected because q_4 finds the second b. Then in the simulator, runs 1 and 2 differ by exactly one b and give accept and reject, so the count is exact in both directions.

Q3 - Binary Transformer (V = 2, so the operation is x2)

Task A - Algorithm

Multiplying a binary number by 2 shifts every bit one place to the left, which is the same as writing a 0 at the right end of the number. So the machine doesn't have to move any bits, it just has to get to the right end and write a 0 on the first blank. q0 looks at the first bit, then q1 moves right over the 0s and 1s, and when it reaches the blank it writes 0 and turns around into q3. q3 walks back to the left end and then steps right onto the first digit and halts in q4, so the tape holds only the answer with the head at its start. The special case is the number 0: if the input is just "0" (q0 -> q2 -> blank) nothing is appended, because $0 \times 2 = 0$ and "00" would not be a clean answer. If the tape is empty the machine writes a single 0.

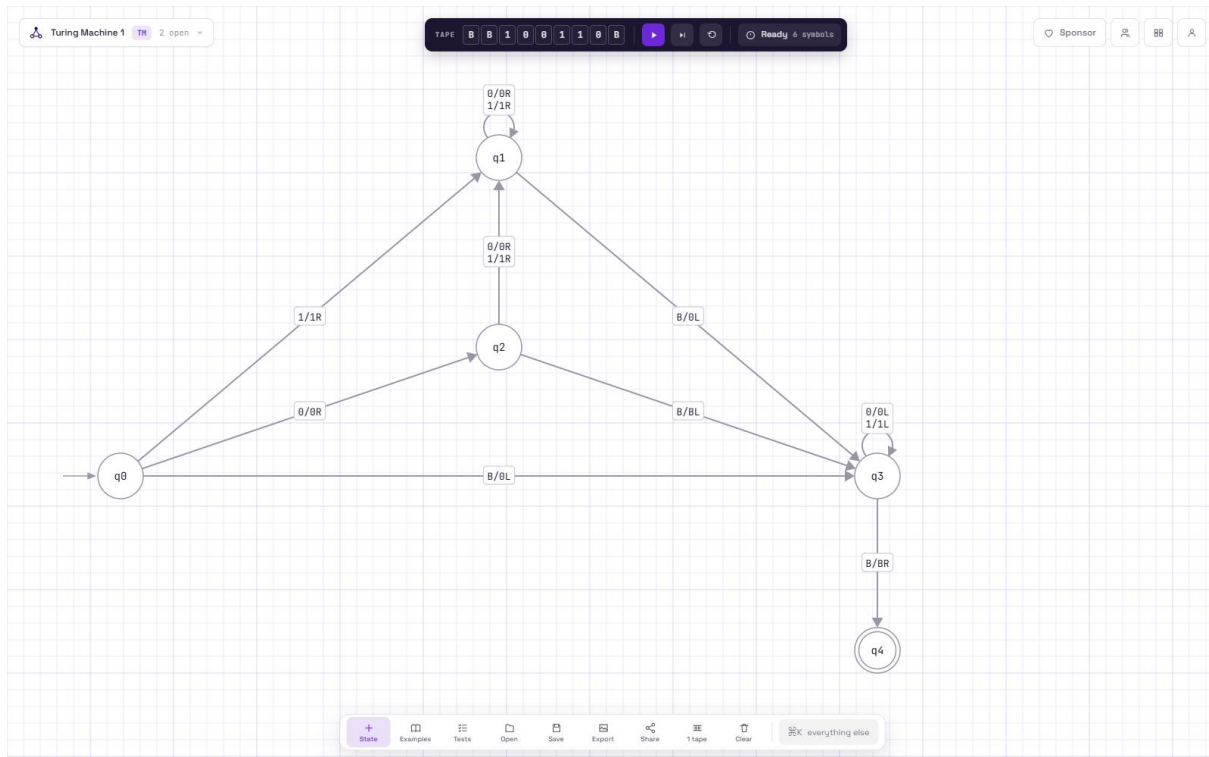
Task B - Transition table and state diagram

State	Meaning
q0 (start)	read the first bit
q1	move right to the end of the number
q2	first bit was 0, check if the number is just "0"
q3	move back left to the start
q4	halt (done)

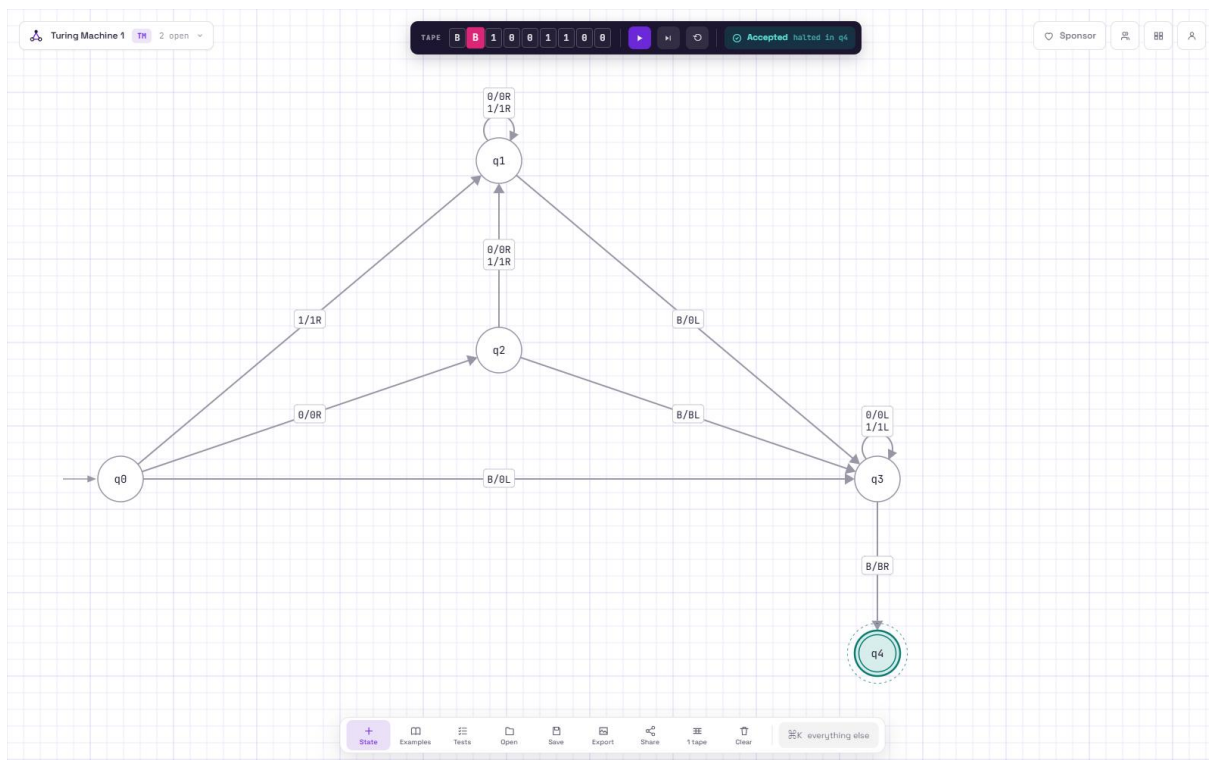
Current state	Read	Write	Move	Next state
q0	1	1	R	q1
q0	0	0	R	q2
q0	B	0	L	q3
q1	0	0	R	q1
q1	1	1	R	q1
q1	B	0	L	q3
q2	0	0	R	q1
q2	1	1	R	q1
q2	B	B	L	q3
q3	0	0	L	q3
q3	1	1	L	q3
q3	B	B	R	q4

Task C - Simulator run

Starting tape: 100110 (my B). Files: Q3/q3_run1_B.json, Q3/q3_run1_B_initial.png (before) and Q3/q3_run1_B.png (after). Link is in README.md (line "Q3 Run 1").



Before: tape holds 100110



After: tape holds 1001100, halted in q4

Working by hand:

$$B = 100110 = 32 + 4 + 2 = 38$$

$$38 \times 2 = 76$$

$$76 = 64 + 8 + 4 = 1001100 \text{ in binary}$$

Final tape in the simulator: 1001100, which is 76. So it matches.

Task D - Edge case

The edge case I had to handle is the input 0. If the machine just appended a 0 every time, 0 would come out as 00, which is the right value but not a clean binary answer, so q2 checks for it and leaves 0 alone. Also the change has to be made at the right end of the number, not the left. If I tried to do it by changing the leftmost bit, for example putting the extra 0 right after the first 1, 100110 would become 1000110 = 70 instead of 1001100 = 76, so the head really has to go all the way to the end first.

Q4 - Palindrome-Family Recognizer ($V = 2$, even-length palindromes over $\{a,b\}$)

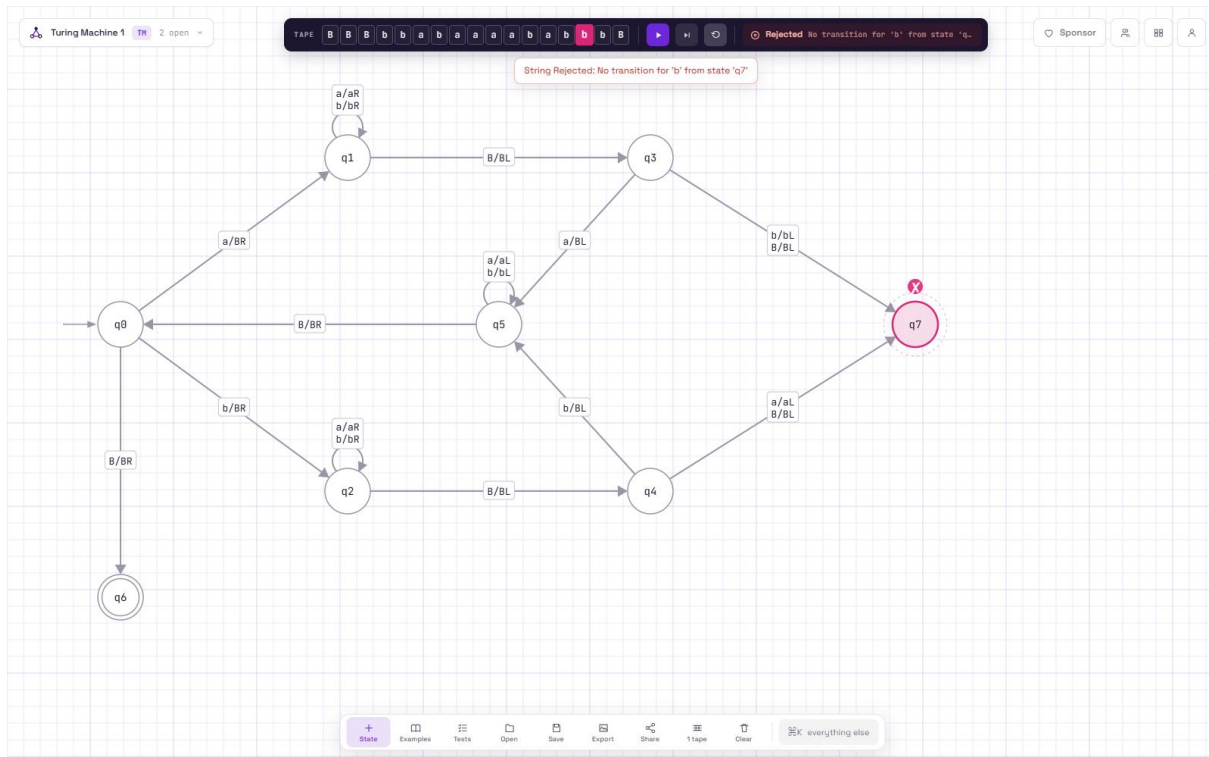
Task A - Strategy

The machine compares the outermost symbols and then works its way inward. In q_0 it reads the leftmost symbol that is left, erases it (writes B) and remembers what it was by going to q_1 for an a or q_2 for a b. It then runs right to the end of the string and steps back onto the last symbol. In q_3 the last symbol has to be an a, and in q_4 it has to be a b. If it matches, it is erased as well and q_5 walks the head back to the left end for the next pair. If it does not match the machine goes to the reject state q_7 . Since only even-length palindromes count, the middle matters: if q_3 or q_4 finds a blank instead of a symbol, the symbol I just erased was alone in the middle, so the length was odd and the machine rejects. When q_0 finds a blank at the start of a round, every pair matched and nothing was left over, so it accepts in q_6 .

Task B - Transition table and state diagram

State	Meaning
q_0 (start)	erase the leftmost symbol and remember it
q_1	remembered a, go to right end
q_2	remembered b, go to right end
q_3	last symbol must be a
q_4	last symbol must be b
q_5	go back to the left end
q_6	accept
q_7	reject

Current state	Read	Write	Move	Next state
q_0	a	B	R	q_1
q_0	b	B	R	q_2
q_0	B	B	R	q_6
q_1	a	a	R	q_1
q_1	b	b	R	q_1
q_1	B	B	L	q_3
q_2	a	a	R	q_2
q_2	b	b	R	q_2
q_2	B	B	L	q_4
q_3	a	B	L	q_5
q_3	b	b	L	q_7
q_3	B	B	L	q_7
q_4	b	B	L	q_5
q_4	a	a	L	q_7
q_4	B	B	L	q_7
q_5	a	a	L	q_5
q_5	b	b	L	q_5
q_5	B	B	R	q_0



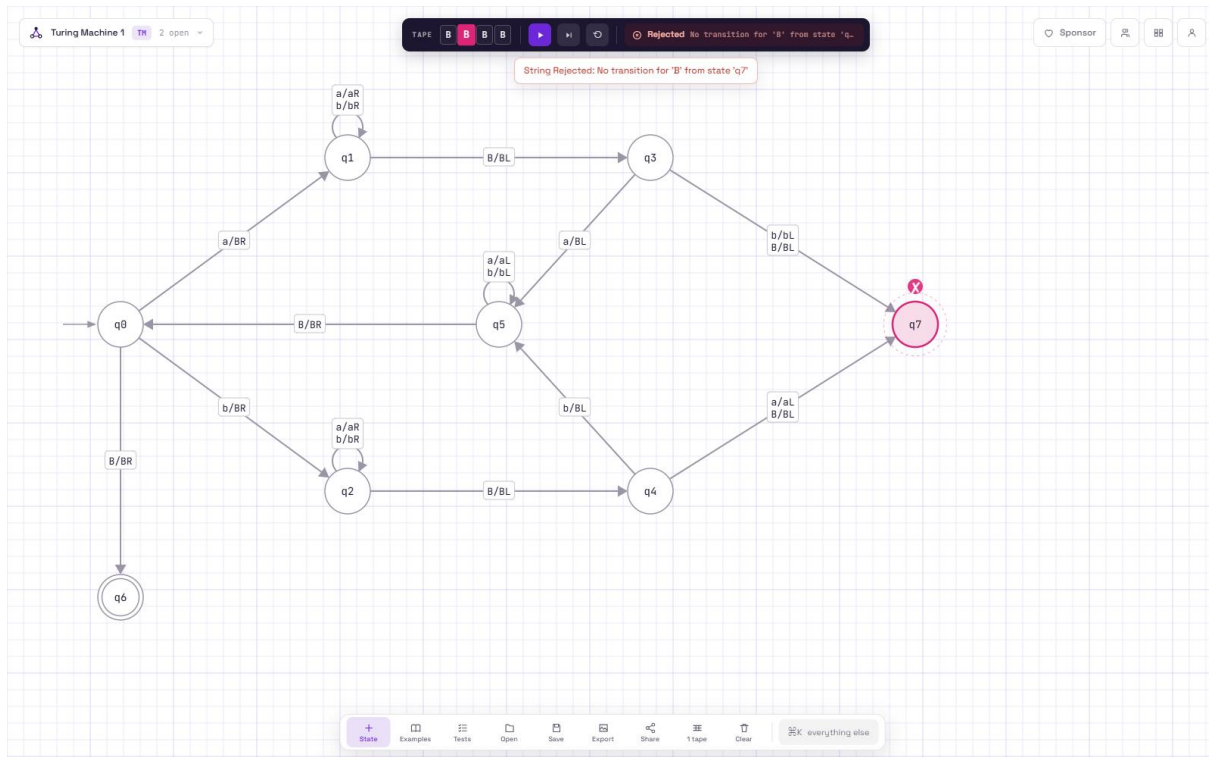
State diagram in AutomataVerse (screenshot from run 2)

Task C - Simulator runs

$n = 7$ so $2n = 14$. Files are in the Q4 folder and links in README.md (lines "Q4 Run 1/2/3").

Run	Input	Result	Correct?	Files
1	abbabaaaababba (length 14, abbabaa followed by its reverse)	Accepted, halted in q6	Yes	q4_run1_in.json / .png
2	abbabaaaababbb (length 14, last letter changed to b)	Rejected, halted in q7	Yes, first a vs last b	q4_run2_not_in.json / .png
3	a (single symbol)	Rejected, halted in q7	Yes, length 1 is odd	q4_run3_single.json / .png

For run 3 I picked a single symbol because that is the boundary for the even-length version: "a" is a palindrome but has odd length so it has to be rejected, while the empty string (length 0) is accepted straight away by the q0 on B transition.



Run 3: a, rejected

Task D - Why several sweeps are needed

The head can only see one cell at a time and the only thing the machine remembers is its current state. To check the first symbol against the last one it has to carry the first symbol (as **q1** or **q2**) all the way to the other end, and then come all the way back to fetch the next symbol, and so on. With only one left-to-right pass the machine would have to remember the whole first half of the string in its states before it could compare anything, and since the string can be any length that would need an unlimited number of states. Erasing the matched pair and sweeping back and forth lets the tape do the remembering. For my 14 letter input that was 7 round trips.

Q5 - Marked-Comparison Recognizer ($V = 2, L = \{ a^i \# b^j : i > j \}$)

Task A - Strategy

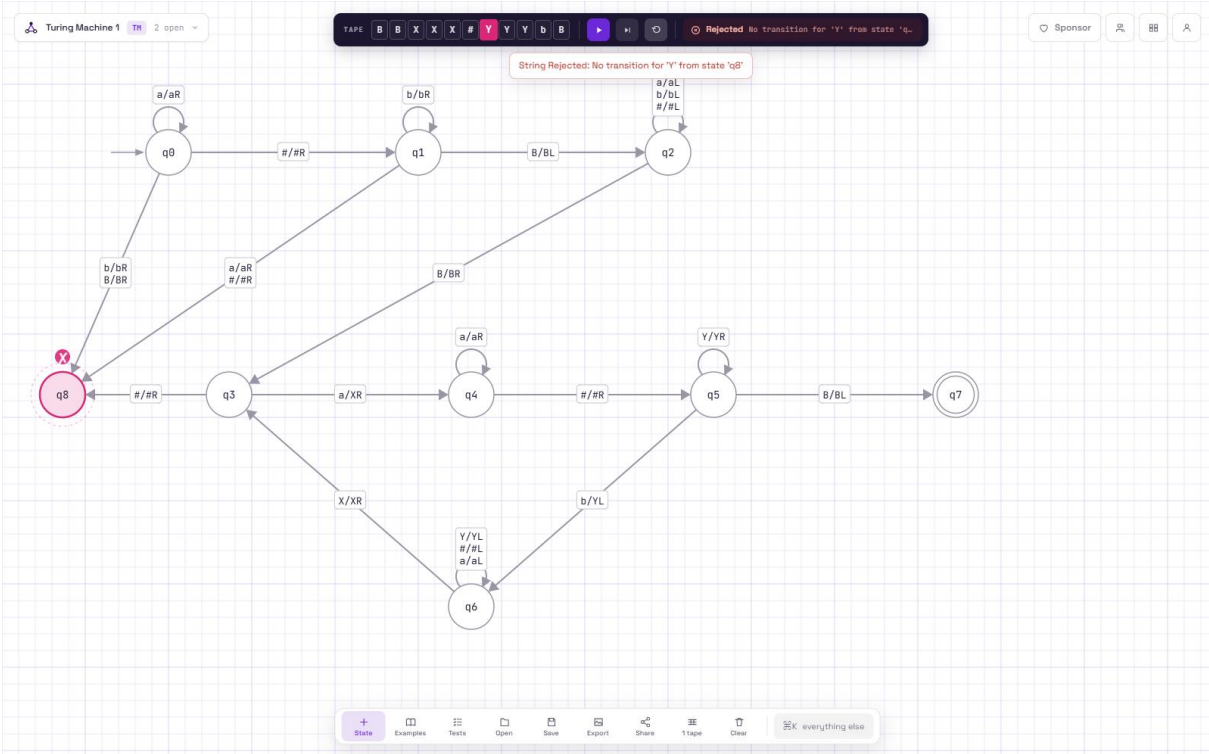
My machine works in two phases. First, q_0, q_1 and q_2 check the format: some a's, then one #, then some b's, and anything else goes to the reject state q_8 . q_2 then rewinds the head to the left end. In the second phase every round crosses off one a on the left by writing X (q_3 to q_4), moves right past the other a's and the # (q_4), skips the b's already marked as Y (q_5) and marks the first unmarked b as Y. q_6 walks back left until it reaches an X and steps right into q_3 for the next round. The loop can end in two ways. If q_5 is looking for a b to pair with the a it just crossed and finds the blank, that a has no partner, so $i > j$ and the machine accepts in q_7 . If q_3 is looking for the next a and finds the # instead, every a got a b, so $i \leq j$ and it rejects in q_8 .

Task B - Transition table and state diagram

State	Meaning
q_0 (start)	format check: a's before #
q_1	format check: b's after #
q_2	rewind to the left end
q_3	cross the next a (X)
q_4	move right to the #
q_5	find the next unmarked b and mark it Y
q_6	go back left to the last X
q_7	accept
q_8	reject

Current state	Read	Write	Move	Next state
q_0	a	a	R	q_0
q_0	#	#	R	q_1
q_0	b	b	R	q_8
q_0	B	B	R	q_8
q_1	b	b	R	q_1
q_1	B	B	L	q_2
q_1	a	a	R	q_8
q_1	#	#	R	q_8
q_2	a	a	L	q_2
q_2	b	b	L	q_2
q_2	#	#	L	q_2
q_2	B	B	R	q_3
q_3	a	X	R	q_4
q_3	#	#	R	q_8
q_4	a	a	R	q_4
q_4	#	#	R	q_5
q_5	Y	Y	R	q_5
q_5	b	Y	L	q_6

q5	B	B	L	q7
q6	Y	Y	L	q6
q6	#	#	L	q6
q6	a	a	L	q6
q6	X	X	R	q3



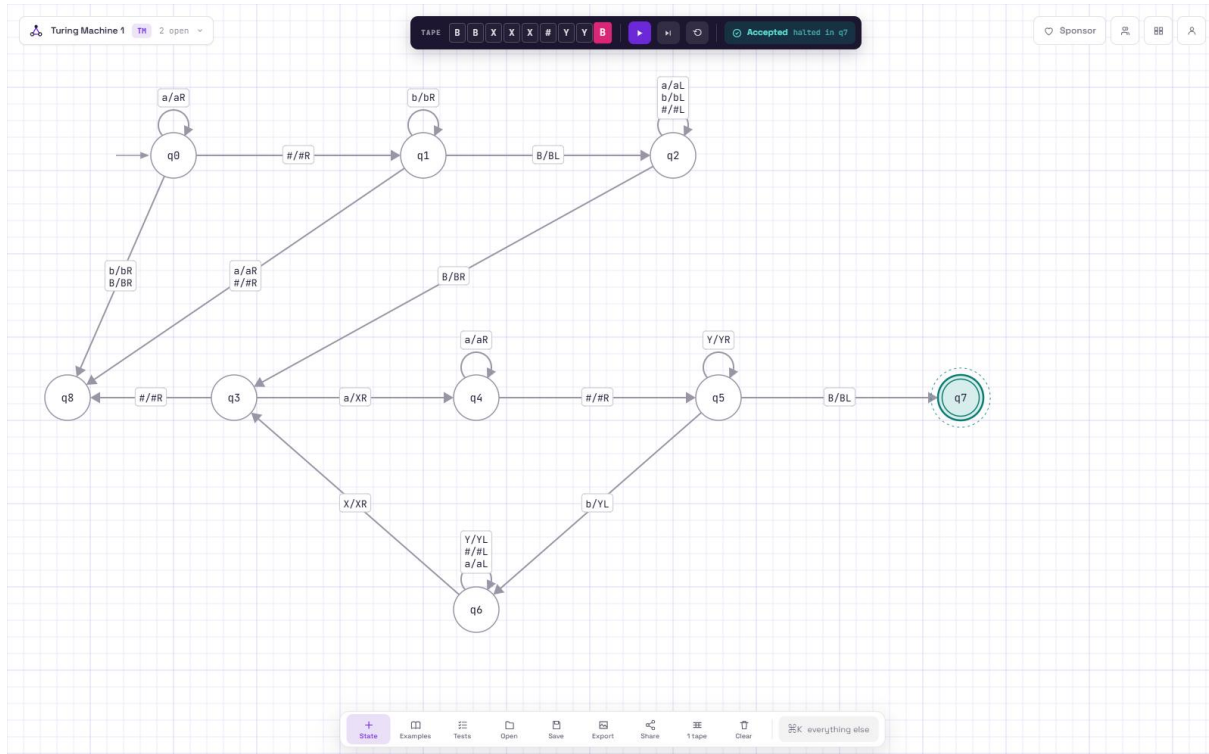
State diagram in AutomataVerse (screenshot from run 2)

Task C - Simulator runs

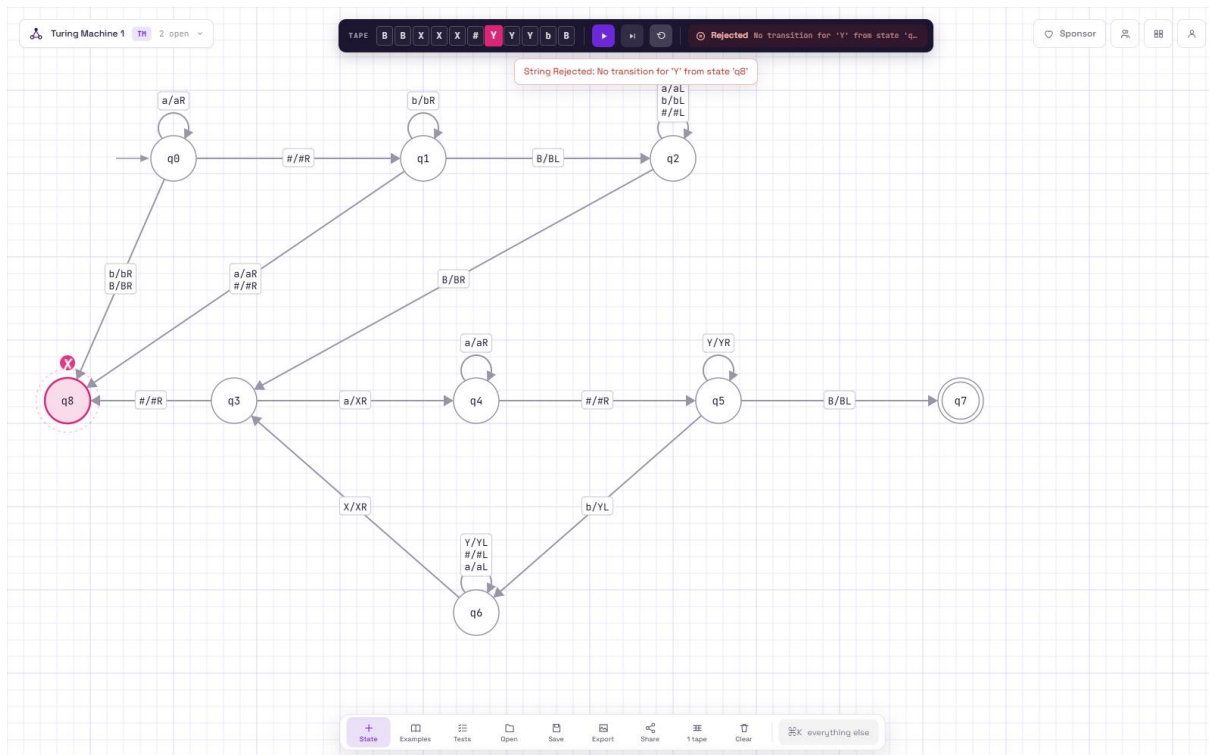
Building the left side from B: B = 100110. Writing an a for every 1-bit and a b for every 0-bit gives a b b a a b, which is three a's (from the 1-bits) and three b's (from the 0-bits). My language only allows a's on the left of the #, so I put the a's from the three 1-bits on the left: aaa, which means $i = 3$. Then I chose the right side so that the first string has more a's than b's and the second one has more b's than a's.

Run	Input	i, j	Result	Correct?	Files
1	aaa#bb	$i = 3, j = 2$	Accepted, halted in q7, tape XXX#YY	Yes, $3 > 2$	q5_run1_in.json / .png
2	aaa#bbbb	$i = 3, j = 4$	Rejected, halted in q8, tape XXX#YYYb	Yes, 3 is not > 4	q5_run2_not_in.json / .png

Links are in README.md (lines "Q5 Run 1" and "Q5 Run 2").



Run 1: aaa#bb, accepted



Run 2: aaa#bbbb, rejected

Task D - Why one pass is not enough

To decide whether $i > j$ the machine has to compare two counts that can be any size. If it only went left to right once, by the time it reached the b's it would have to remember how many a's it had seen, and a finite number of states can only count up to some fixed number, so long enough strings would break it. My machine uses the tape as memory instead: the X's on the left show which a's are already paired and the Y's on the right show which b's are used. The head cannot be on both sides at the same time, so after marking a b on the right (q5) it has to come back left (q6) to find the next a, and that is the back and forth movement.

Q6 - Debug the Broken Machine

Task A - Hand traces

Notation: the cell in [] is where the head is. Cell 0 is the first cell of the input. □ is blank.

Input "" (empty string):

Step	State	Tape	Head at cell	Transition used
0	q_even	[□] □	0	(q_even, □) -> write □, R, q_odd
1	q_odd	□ [□]	1	(q_odd, □) -> write □, R, q_reject
2	q_reject	□ □ [□]	2	halts, REJECT

Input "11":

Step	State	Tape	Head at cell	Transition used
0	q_even	[1] 1 □	0	(q_even, 1) -> write 1, R, q_odd
1	q_odd	1 [1] □	1	(q_odd, 1) -> write 1, R, q_even
2	q_even	1 1 [□]	2	(q_even, □) -> write □, R, q_odd
3	q_odd	1 1 □ [□]	3	(q_odd, □) -> write □, R, q_reject
4	q_reject	1 1 □ □ [□]	4	halts, REJECT

Both inputs end up rejected, but both have an even number of 1s (zero and two), so they should be accepted.

Task B - Finding the wrong row

The wrong row is: q_even, □, □, R, q_odd.

In both traces the machine was in q_even at the exact moment it first read a blank (step 0 for "" and step 2 for "11"). Being in q_even at the blank means the whole input has been read and the number of 1s was even, so this is the point where it should accept. Instead that row sends it to q_odd, even though no 1 was read, and then q_odd reads another blank and goes to q_reject. So both wrong answers came from the same row. The other rows behaved correctly in the traces: 0 keeps the state, 1 flips between q_even and q_odd, and q_odd on blank going to q_reject is right because an odd count must be rejected. Another hint is that nothing in the table ever goes to q_accept, so the machine could never accept anything.

Corrected row: q_even, □, □, R, q_accept.

Task C - Simulator runs

AutomataVerse always uses q0 as the start state, so I named the states like this: q0 = q_even, q1 = q_odd, q2 = q_accept, q3 = q_reject. The blank is B.

Broken machine (exactly as given):

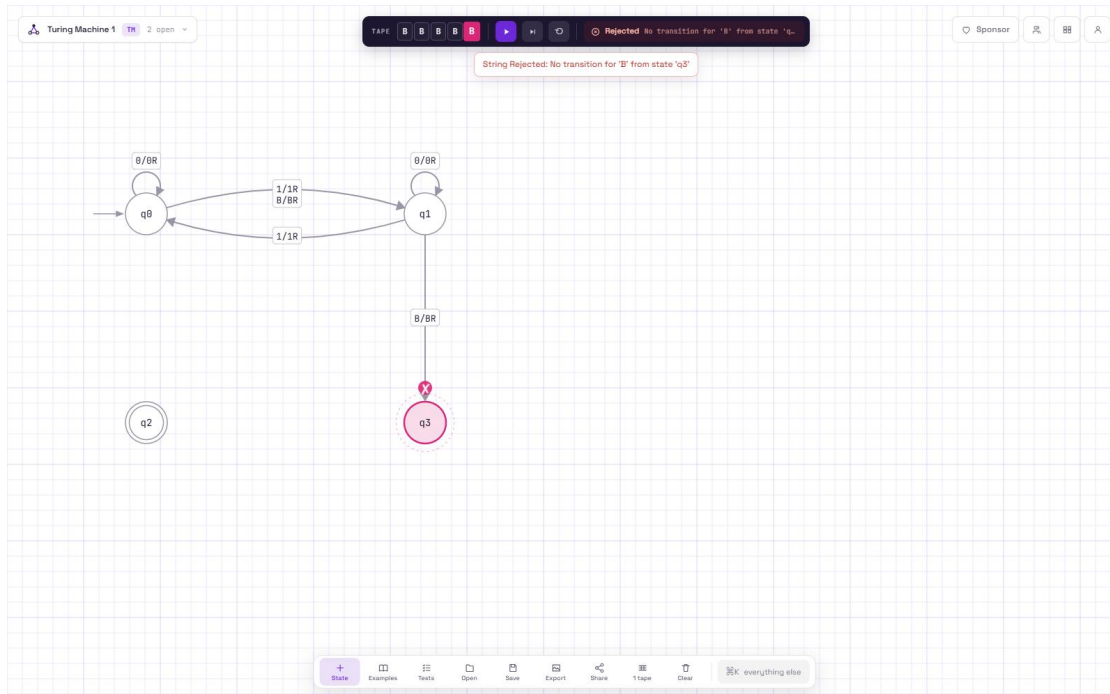
Current state	Read	Write	Move	Next state
q0 (q_even)	0	0	R	q0 (q_even)
q0 (q_even)	1	1	R	q1 (q_odd)
q0 (q_even)	B	B	R	q1 (q_odd)
q1 (q_odd)	0	0	R	q1 (q_odd)
q1 (q_odd)	1	1	R	q0 (q_even)
q1 (q_odd)	B	B	R	q3 (q_reject)

Fixed machine (only the q0 on B row changed):

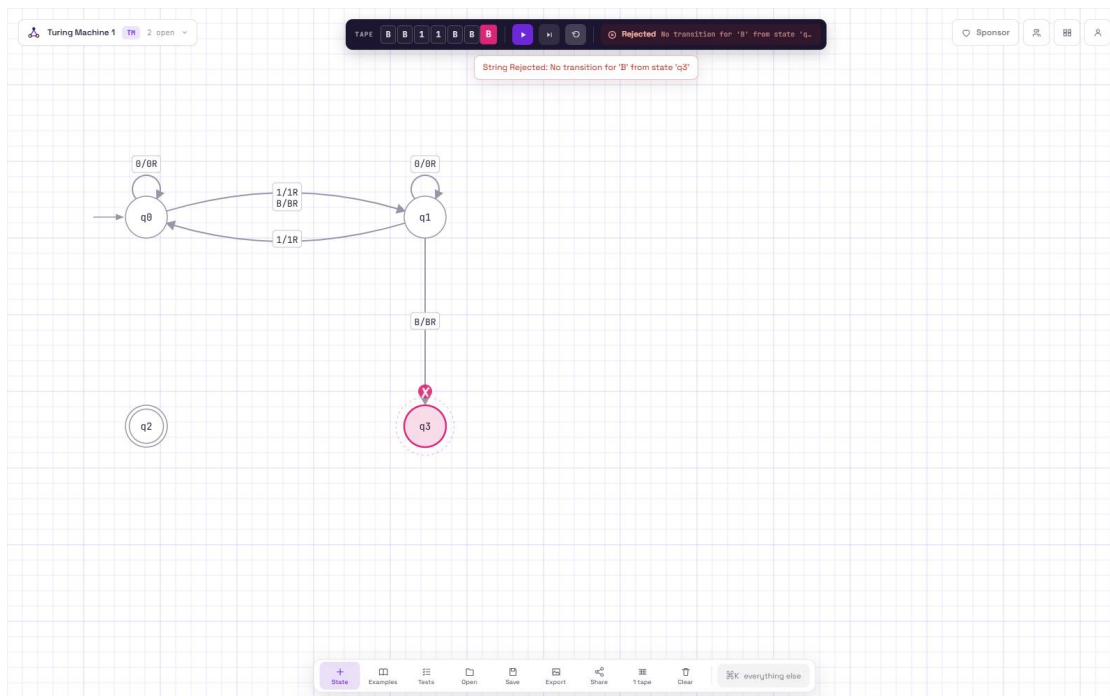
Current state	Read	Write	Move	Next state
q0 (q_even)	0	0	R	q0 (q_even)
q0 (q_even)	1	1	R	q1 (q_odd)
q0 (q_even)	B	B	R	q2 (q_accept)
q1 (q_odd)	0	0	R	q1 (q_odd)
q1 (q_odd)	1	1	R	q0 (q_even)
q1 (q_odd)	B	B	R	q3 (q_reject)

Summary of the eight runs (links in README.md, lines "Q6 Broken Run 1-4" and "Q6 Fixed Run 1-4"):

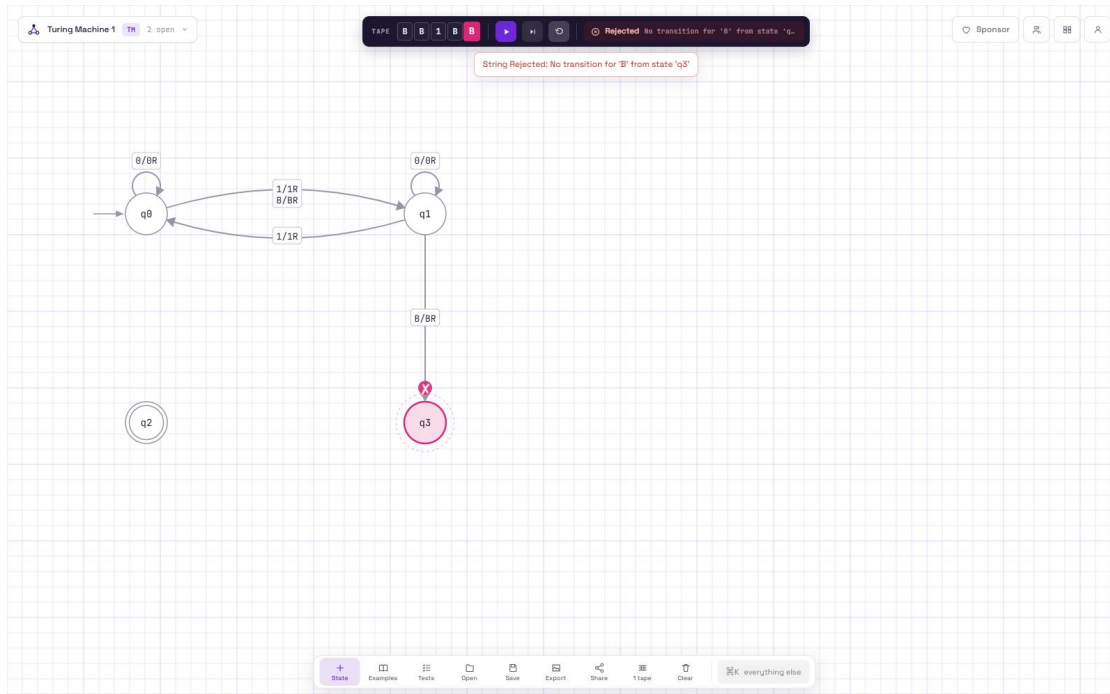
String	Broken	Fixed	Correct answer
"" (empty)	Reject	Accept	Accept (zero 1s)
11	Reject	Accept	Accept (two 1s)
1	Reject	Reject	Reject (one 1)
111	Reject	Reject	Reject (three 1s)



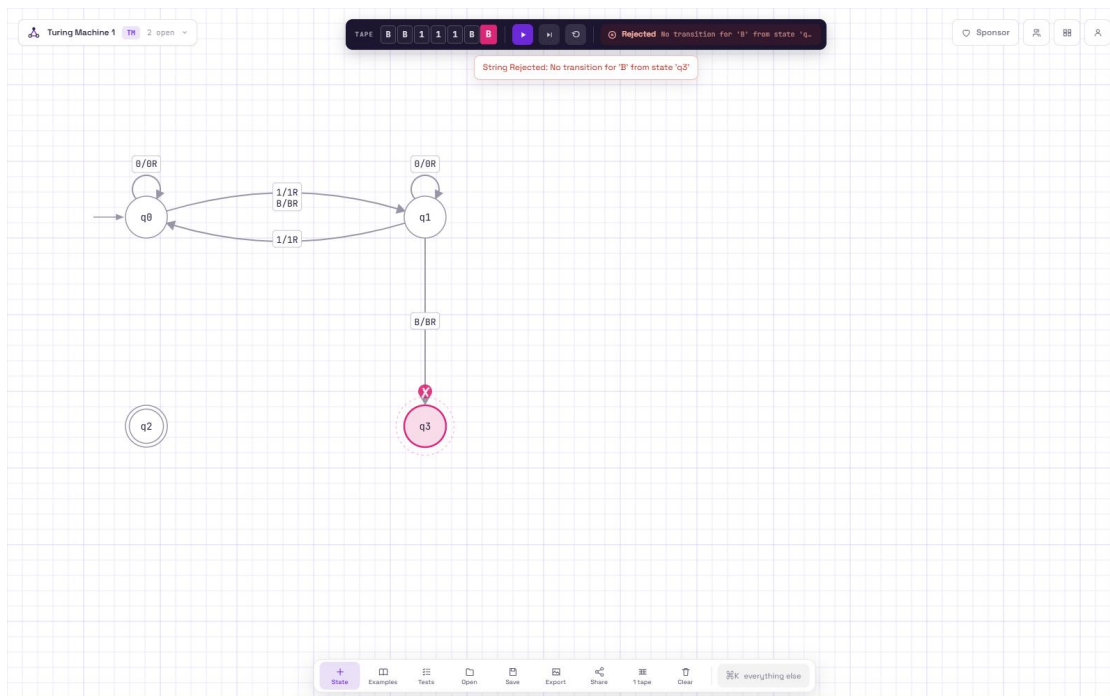
Broken machine, run 1: input "" rejected (q6_broken_run1_empty.json)



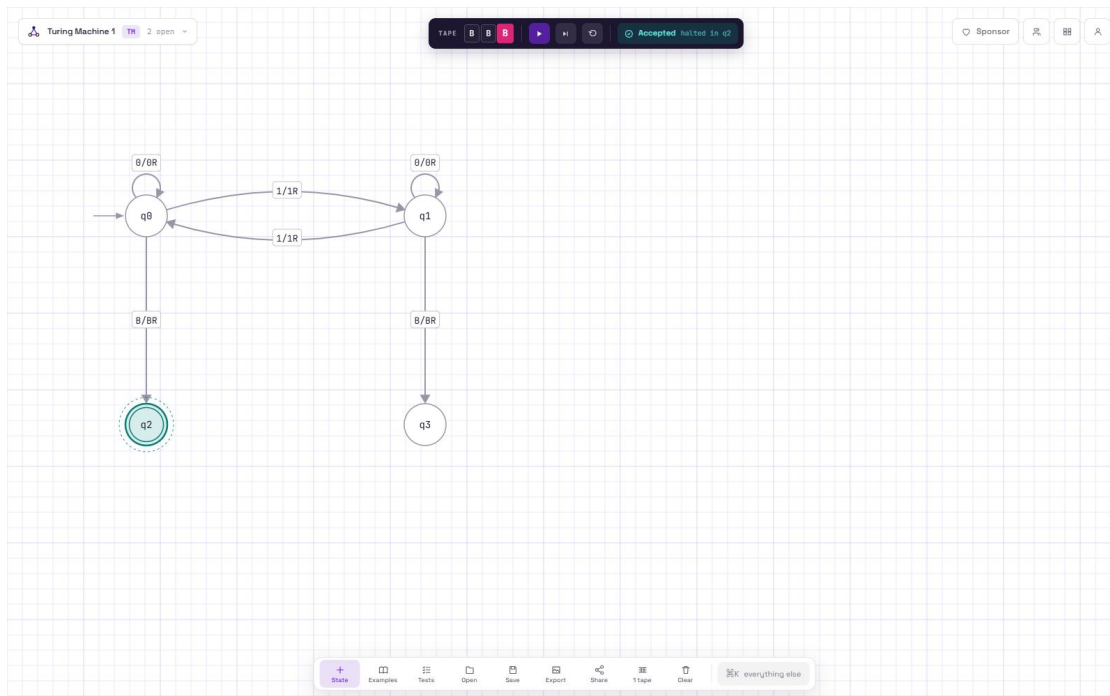
Broken machine, run 2: input "11" rejected (q6_broken_run2_11.json)



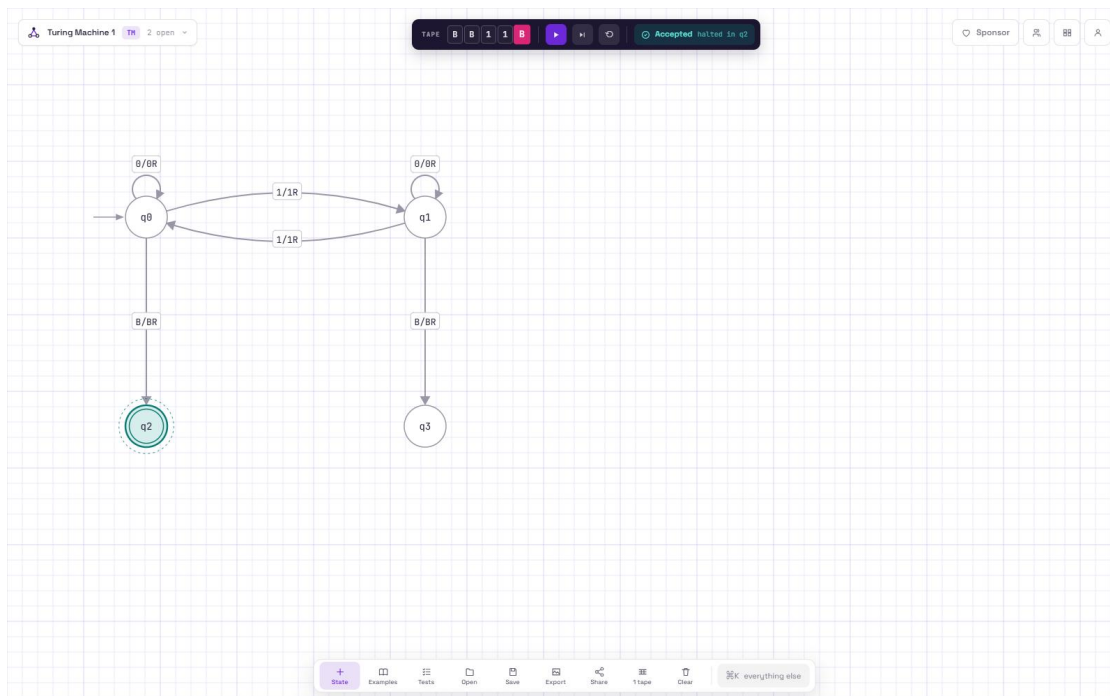
Broken machine, run 3: input "1" rejected (q6_broken_run3_1.json)



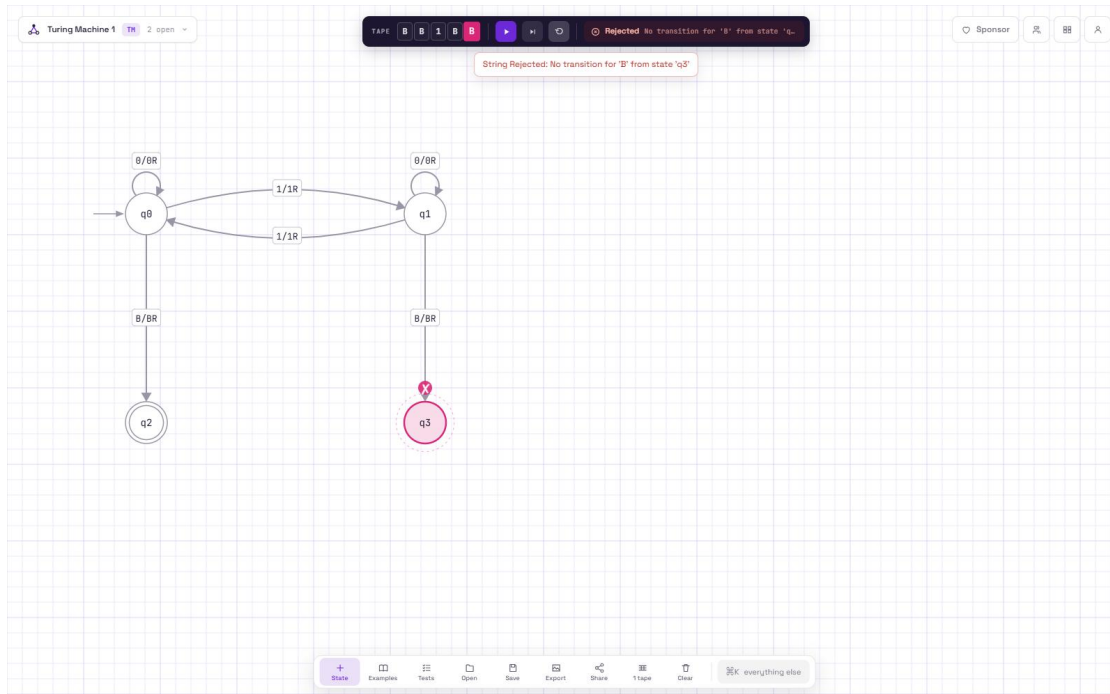
Broken machine, run 4: input "111" rejected (q6_broken_run4_111.json)



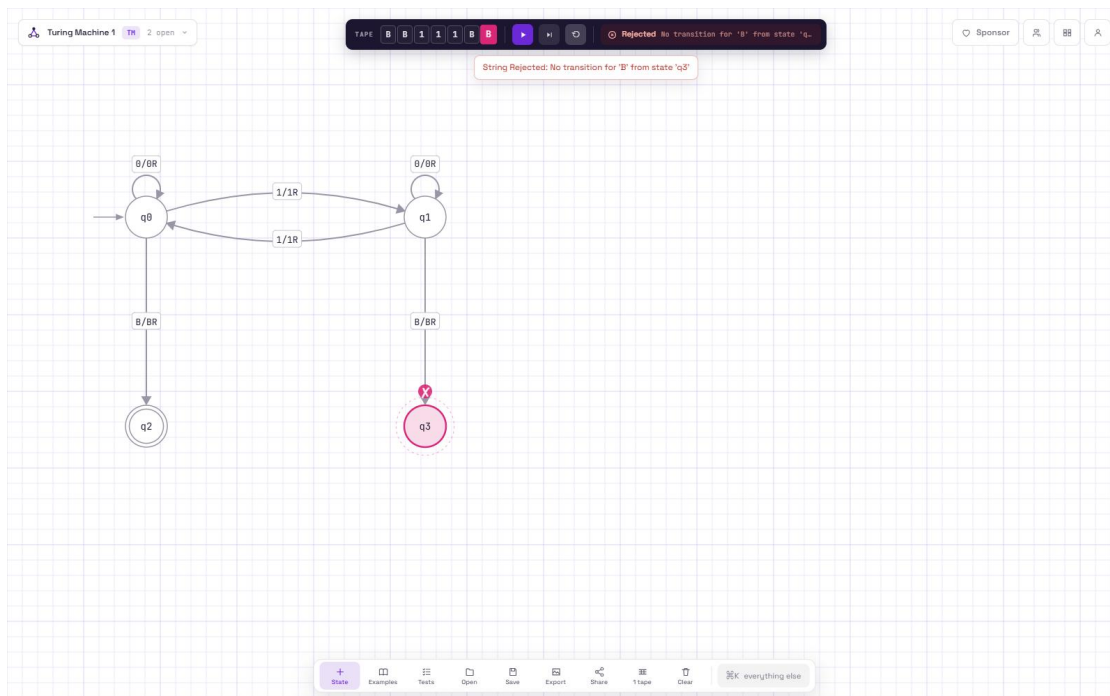
Fixed machine, run 1: input "" accepted (q6_fixed_run1_empty.json)



Fixed machine, run 2: input "11" accepted (q6_fixed_run2_11.json)



Fixed machine, run 3: input "1" rejected (q6_fixed_run3_1.json)



Fixed machine, run 4: input "111" rejected (q6_fixed_run4_111.json)

Task D - Reflection

The strings 1 and 111 on their own would not have shown the bug. Both have an odd number of 1s so the right answer is reject, and the broken machine rejects them too because it reaches the blank in q_{odd} , which uses a correct row. The bug only shows up on inputs that reach the blank in q_{even} , meaning strings that should be accepted, like "" and 11. So when I test my own machines I should not only try strings that should be rejected, I should make sure there is at least one test that ends in each halting row, including the accepting one, and always try edge cases like the empty string.

Appendix - Machine JSON (AutomataVerse export)

These are the JSON files exported from AutomataVerse with the Save button. Every run of the same machine uses the same states and transitions, only "inputString" changes, so one file per machine is shown here. The rest are in the repository.

Q1 - divisible by 5 (Q1/q1_run1_B.json)

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 200,
      "y": 350
    },
    "q1": {
      "id": "q1",
      "x": 500,
      "y": 200
    },
    "q2": {
      "id": "q2",
      "x": 800,
      "y": 350
    },
    "q3": {
      "id": "q3",
      "x": 500,
      "y": 560
    },
    "q4": {
      "id": "q4",
      "x": 1100,
      "y": 350
    },
    "q5": {
      "id": "q5",
      "x": 200,
      "y": 720
    },
    "q6": {
      "id": "q6",
      "x": 1100,
      "y": 720
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "0/0R",
        "sourceid": "q0",
        "targetid": "q0"
      },
      {
        "label": "1/1R",
        "sourceid": "q0",
        "targetid": "q1"
      }
    ],

```

```
{
  "label": "B/BL",
  "sourceid": "q0",
  "targetid": "q5"
},
"q1": [
  {
    "label": "0/0R",
    "sourceid": "q1",
    "targetid": "q2"
  },
  {
    "label": "1/1R",
    "sourceid": "q1",
    "targetid": "q3"
  },
  {
    "label": "B/BL",
    "sourceid": "q1",
    "targetid": "q6"
  }
],
"q2": [
  {
    "label": "0/0R",
    "sourceid": "q2",
    "targetid": "q4"
  },
  {
    "label": "1/1R",
    "sourceid": "q2",
    "targetid": "q0"
  },
  {
    "label": "B/BL",
    "sourceid": "q2",
    "targetid": "q6"
  }
],
"q3": [
  {
    "label": "0/0R",
    "sourceid": "q3",
    "targetid": "q1"
  },
  {
    "label": "1/1R",
    "sourceid": "q3",
    "targetid": "q2"
  },
  {
    "label": "B/BL",
    "sourceid": "q3",
    "targetid": "q6"
  }
],
"q4": [
  {
    "label": "0/0R",
```

```

        "sourceid": "q4",
        "targetid": "q3"
    },
    {
        "label": "1/1R",
        "sourceid": "q4",
        "targetid": "q4"
    },
    {
        "label": "B/BL",
        "sourceid": "q4",
        "targetid": "q6"
    }
]
},
"finalNodes": [
    "q5"
],
"nodeNum": 7,
"automataType": "TM",
"inputString": "100110",
"stageProps": {
    "x": 0,
    "y": 0,
    "scale": 1,
    "draggable": true
},
"stateOutput": {},
"tapeMode": "single",
"inputString2": ""
}

```

Q2 - $a^{2n} b^n$ (Q2/q2_run1_n7.json)

```

{
    "nodeMap": {
        "q0": {
            "id": "q0",
            "x": 200,
            "y": 330
        },
        "q1": {
            "id": "q1",
            "x": 500,
            "y": 330
        },
        "q2": {
            "id": "q2",
            "x": 850,
            "y": 330
        },
        "q3": {
            "id": "q3",
            "x": 500,
            "y": 600
        },
        "q4": {
            "id": "q4",
            "x": 200,

```

```

    "y": 650
  },
  "q5": {
    "id": "q5",
    "x": 480,
    "y": 820
  },
  "q6": {
    "id": "q6",
    "x": 500,
    "y": 130
  }
},
"transitionMap": {
  "q0": [
    {
      "label": "a/XR",
      "sourceid": "q0",
      "targetid": "q1"
    },
    {
      "label": "Y/YR",
      "sourceid": "q0",
      "targetid": "q4"
    },
    {
      "label": "B/BR",
      "sourceid": "q0",
      "targetid": "q5"
    },
    {
      "label": "b/bR",
      "sourceid": "q0",
      "targetid": "q6"
    }
  ],
  "q1": [
    {
      "label": "a/XR",
      "sourceid": "q1",
      "targetid": "q2"
    },
    {
      "label": "b/bR,Y/YR,B/BR",
      "sourceid": "q1",
      "targetid": "q6"
    }
  ],
  "q2": [
    {
      "label": "a/aR,Y/YR",
      "sourceid": "q2",
      "targetid": "q2"
    },
    {
      "label": "b/YL",
      "sourceid": "q2",
      "targetid": "q3"
    }
  ],
  {

```

```

        "label": "B/BR",
        "sourceid": "q2",
        "targetid": "q6"
    }
],
"q3": [
    {
        "label": "a/aL,Y/YL",
        "sourceid": "q3",
        "targetid": "q3"
    },
    {
        "label": "X/XR",
        "sourceid": "q3",
        "targetid": "q0"
    }
],
"q4": [
    {
        "label": "Y/YR",
        "sourceid": "q4",
        "targetid": "q4"
    },
    {
        "label": "B/BR",
        "sourceid": "q4",
        "targetid": "q5"
    },
    {
        "label": "a/aR,b/bR",
        "sourceid": "q4",
        "targetid": "q6"
    }
]
},
"finalNodes": [
    "q5"
],
"nodeNum": 7,
"automataType": "TM",
"inputString": "aaaaaaaaaaaaabbbbbbb",
"stageProps": {
    "x": 0,
    "y": 0,
    "scale": 1,
    "draggable": true
},
"stateOutput": {},
"tapeMode": "single",
"inputString2": ""
}

```

Q3 - x2 (Q3/q3_run1_B.json)

```

{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 150,

```

```

    "y": 620
  },
  "q1": {
    "id": "q1",
    "x": 650,
    "y": 200
  },
  "q2": {
    "id": "q2",
    "x": 650,
    "y": 450
  },
  "q3": {
    "id": "q3",
    "x": 1150,
    "y": 620
  },
  "q4": {
    "id": "q4",
    "x": 1150,
    "y": 850
  }
},
"transitionMap": {
  "q0": [
    {
      "label": "1/1R",
      "sourceid": "q0",
      "targetid": "q1"
    },
    {
      "label": "0/0R",
      "sourceid": "q0",
      "targetid": "q2"
    },
    {
      "label": "B/0L",
      "sourceid": "q0",
      "targetid": "q3"
    }
  ],
  "q2": [
    {
      "label": "0/0R,1/1R",
      "sourceid": "q2",
      "targetid": "q1"
    },
    {
      "label": "B/BL",
      "sourceid": "q2",
      "targetid": "q3"
    }
  ],
  "q1": [
    {
      "label": "0/0R,1/1R",
      "sourceid": "q1",
      "targetid": "q1"
    },
    {

```

```

        "label": "B/0L",
        "sourceid": "q1",
        "targetid": "q3"
    }
],
"q3": [
    {
        "label": "0/0L,1/1L",
        "sourceid": "q3",
        "targetid": "q3"
    },
    {
        "label": "B/BR",
        "sourceid": "q3",
        "targetid": "q4"
    }
]
},
"finalNodes": [
    "q4"
],
"nodeNum": 5,
"automataType": "TM",
"inputString": "100110",
"stageProps": {
    "x": 0,
    "y": 0,
    "scale": 1,
    "draggable": true
},
"stateOutput": {},
"tapeMode": "single",
"inputString2": ""
}

```

Q4 - even-length palindromes (Q4/q4_run1_in.json)

```

{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 150,
      "y": 420
    },
    "q1": {
      "id": "q1",
      "x": 450,
      "y": 200
    },
    "q2": {
      "id": "q2",
      "x": 450,
      "y": 640
    },
    "q3": {
      "id": "q3",
      "x": 850,
      "y": 200
    },
  },

```

```
"q4": {
  "id": "q4",
  "x": 850,
  "y": 640
},
"q5": {
  "id": "q5",
  "x": 650,
  "y": 420
},
"q6": {
  "id": "q6",
  "x": 150,
  "y": 780
},
"q7": {
  "id": "q7",
  "x": 1200,
  "y": 420
}
},
"transitionMap": {
  "q0": [
    {
      "label": "a/BR",
      "sourceid": "q0",
      "targetid": "q1"
    },
    {
      "label": "b/BR",
      "sourceid": "q0",
      "targetid": "q2"
    },
    {
      "label": "B/BR",
      "sourceid": "q0",
      "targetid": "q6"
    }
  ],
  "q1": [
    {
      "label": "a/aR,b/bR",
      "sourceid": "q1",
      "targetid": "q1"
    },
    {
      "label": "B/BL",
      "sourceid": "q1",
      "targetid": "q3"
    }
  ],
  "q2": [
    {
      "label": "a/aR,b/bR",
      "sourceid": "q2",
      "targetid": "q2"
    },
    {
      "label": "B/BL",
      "sourceid": "q2",
```

```

        "targetid": "q4"
    }
],
"q3": [
    {
        "label": "a/BL",
        "sourceid": "q3",
        "targetid": "q5"
    },
    {
        "label": "b/bL,B/BL",
        "sourceid": "q3",
        "targetid": "q7"
    }
],
"q4": [
    {
        "label": "b/BL",
        "sourceid": "q4",
        "targetid": "q5"
    },
    {
        "label": "a/aL,B/BL",
        "sourceid": "q4",
        "targetid": "q7"
    }
],
"q5": [
    {
        "label": "a/aL,b/bL",
        "sourceid": "q5",
        "targetid": "q5"
    },
    {
        "label": "B/BR",
        "sourceid": "q5",
        "targetid": "q0"
    }
]
],
"finalNodes": [
    "q6"
],
"nodeNum": 8,
"automataType": "TM",
"inputString": "abbabaaaababba",
"stageProps": {
    "x": 0,
    "y": 0,
    "scale": 1,
    "draggable": true
},
"stateOutput": {},
"tapeMode": "single",
"inputString2": ""
}

```

Q5 - $a^i \# b^j$, $i > j$ (Q5/q5_run1_in.json)

```

{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 220,
      "y": 200
    },
    "q1": {
      "id": "q1",
      "x": 550,
      "y": 200
    },
    "q2": {
      "id": "q2",
      "x": 880,
      "y": 200
    },
    "q3": {
      "id": "q3",
      "x": 300,
      "y": 520
    },
    "q4": {
      "id": "q4",
      "x": 620,
      "y": 520
    },
    "q5": {
      "id": "q5",
      "x": 940,
      "y": 520
    },
    "q6": {
      "id": "q6",
      "x": 620,
      "y": 800
    },
    "q7": {
      "id": "q7",
      "x": 1260,
      "y": 520
    },
    "q8": {
      "id": "q8",
      "x": 80,
      "y": 520
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "a/aR",
        "sourceid": "q0",
        "targetid": "q0"
      },
      {
        "label": "#/#R",
        "sourceid": "q0",
        "targetid": "q1"
      }
    ],
  },

```

```

    {
      "label": "b/bR,B/BR",
      "sourceid": "q0",
      "targetid": "q8"
    }
  ],
  "q1": [
    {
      "label": "b/bR",
      "sourceid": "q1",
      "targetid": "q1"
    },
    {
      "label": "B/BL",
      "sourceid": "q1",
      "targetid": "q2"
    },
    {
      "label": "a/aR,#/#R",
      "sourceid": "q1",
      "targetid": "q8"
    }
  ],
  "q2": [
    {
      "label": "a/aL,b/bL,#/#L",
      "sourceid": "q2",
      "targetid": "q2"
    },
    {
      "label": "B/BR",
      "sourceid": "q2",
      "targetid": "q3"
    }
  ],
  "q3": [
    {
      "label": "a/XR",
      "sourceid": "q3",
      "targetid": "q4"
    },
    {
      "label": "#/#R",
      "sourceid": "q3",
      "targetid": "q8"
    }
  ],
  "q4": [
    {
      "label": "a/aR",
      "sourceid": "q4",
      "targetid": "q4"
    },
    {
      "label": "#/#R",
      "sourceid": "q4",
      "targetid": "q5"
    }
  ],
  "q5": [

```

```

    {
      "label": "Y/YR",
      "sourceid": "q5",
      "targetid": "q5"
    },
    {
      "label": "b/YL",
      "sourceid": "q5",
      "targetid": "q6"
    },
    {
      "label": "B/BL",
      "sourceid": "q5",
      "targetid": "q7"
    }
  ],
  "q6": [
    {
      "label": "Y/YL, #/#L, a/aL",
      "sourceid": "q6",
      "targetid": "q6"
    },
    {
      "label": "X/XR",
      "sourceid": "q6",
      "targetid": "q3"
    }
  ]
},
"finalNodes": [
  "q7"
],
"nodeNum": 9,
"automataType": "TM",
"inputString": "aaa#bb",
"stageProps": {
  "x": 0,
  "y": 0,
  "scale": 1,
  "draggable": true
},
"stateOutput": {},
"tapeMode": "single",
"inputString2": ""
}

```

Q6 - broken machine (Q6/q6_broken_run2_11.json)

```

{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 200,
      "y": 300
    },
    "q1": {
      "id": "q1",
      "x": 600,
      "y": 300
    }
  }
}

```

```

    },
    "q2": {
      "id": "q2",
      "x": 200,
      "y": 600
    },
    "q3": {
      "id": "q3",
      "x": 600,
      "y": 600
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "0/0R",
        "sourceid": "q0",
        "targetid": "q0"
      },
      {
        "label": "1/1R,B/BR",
        "sourceid": "q0",
        "targetid": "q1"
      }
    ],
    "q1": [
      {
        "label": "0/0R",
        "sourceid": "q1",
        "targetid": "q1"
      },
      {
        "label": "1/1R",
        "sourceid": "q1",
        "targetid": "q0"
      },
      {
        "label": "B/BR",
        "sourceid": "q1",
        "targetid": "q3"
      }
    ]
  },
  "finalNodes": [
    "q2"
  ],
  "nodeNum": 4,
  "automataType": "TM",
  "inputString": "11",
  "stageProps": {
    "x": 0,
    "y": 0,
    "scale": 1,
    "draggable": true
  },
  "stateOutput": {},
  "tapeMode": "single",
  "inputString2": ""
}

```

Q6 - fixed machine (Q6/q6_fixed_run2_11.json)

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 200,
      "y": 300
    },
    "q1": {
      "id": "q1",
      "x": 600,
      "y": 300
    },
    "q2": {
      "id": "q2",
      "x": 200,
      "y": 600
    },
    "q3": {
      "id": "q3",
      "x": 600,
      "y": 600
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "0/0R",
        "sourceid": "q0",
        "targetid": "q0"
      },
      {
        "label": "1/1R",
        "sourceid": "q0",
        "targetid": "q1"
      },
      {
        "label": "B/BR",
        "sourceid": "q0",
        "targetid": "q2"
      }
    ],
    "q1": [
      {
        "label": "0/0R",
        "sourceid": "q1",
        "targetid": "q1"
      },
      {
        "label": "1/1R",
        "sourceid": "q1",
        "targetid": "q0"
      },
      {
        "label": "B/BR",
        "sourceid": "q1",
        "targetid": "q3"
      }
    ]
  }
}
```

```
]
},
"finalNodes": [
  "q2"
],
"nodeNum": 4,
"automataType": "TM",
"inputString": "11",
"stageProps": {
  "x": 0,
  "y": 0,
  "scale": 1,
  "draggable": true
},
"stateOutput": {},
"tapeMode": "single",
"inputString2": ""
}
```